Emotion-Aware Multimodal Nostalgic Photo Styling: A Rule-Based Web System for Caption–Image Contextualization

Oviya Ragothamana, Sushma Pokala, Sammangi Gopi Chand, Raghu Paladi
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, India
Trainees, LTIMindtree
Corresponding author: [Email]

Implementation-based system paper. Results are limited to repository-verifiable behavior; no trained model benchmark or user study is claimed.

Abstract

This paper presents MemoriaAI, a web application that transforms a user-provided photograph and caption into a themed visual memory artifact. The implemented pipeline uses global image statistics and a keyword-based text analyzer to derive coarse affect labels, applies explicit rules to fuse the modalities, computes a bounded nostalgia cue score, chooses a film-inspired preset, and returns a processed image with a template-generated story and heuristic explanation fields. The application is built as a React/Vite frontend and a FastAPI backend, with Pillow and NumPy image processing, Pydantic schemas, and SQLAlchemy history storage. Six presets implement channel-level grading, contrast and saturation changes, optional blur blending, Gaussian noise, and radial vignetting. A browser-side fallback provides a simplified workflow and local history when the API is unavailable.

The repository also contains GoEmotions text files, FER-style image folders, analysis scripts, and API contract tests. These assets are not connected to training or runtime model inference: no learned application checkpoint, training script, or dataset-linked predictions are present. Runtime visual affect is based on RGB mean, channel variation, brightness, and warmth thresholds; runtime text affect is based on fixed keyword weights. The explanation service does not calculate SHAP values or gradient-derived Grad-CAM; it emits lexicon weights and a synthetic Gaussian heatmap. The available tests check endpoint behavior but do not measure predictive quality. Accordingly, accuracy, precision, recall, F1, confusion matrices, comparative performance, and user outcomes are not reported as experimental findings. The work contributes an inspectable software prototype for caption-conditioned nostalgic photo styling and a repository-grounded assessment of its validation requirements.

Keywords

Multimodal affect, nostalgic photo styling, rule-based fusion, image processing, emotion analysis, GoEmotions, explainable interfaces.

1. Introduction

1.1 Background: Photographs shared on social platforms often depend on captions to convey their personal meaning. A reunion image can show smiles while its caption emphasizes elapsed time, gratitude, or longing. Nostalgia research characterizes this experience as self-relevant and socially grounded, often positive but also ambivalent [1]–[3]. Consequently, an image-editing tool that considers both visual appearance and caption context can offer a more situated creative workflow than a filter selected without reference to the accompanying memory.

MemoriaAI is a prototype for this task. A user supplies an image and caption; the application computes simple affect cues, combines them into a named memory category, selects a photographic preset, and returns a styled image alongside a score, story, and explanation panel. Its operation is a deterministic rule system with some stochastic output from image grain and story-template choice. It is not a trained end-to-end multimodal network.

1.2 Problem Statement: The project addresses the engineering problem of connecting caption context to photo styling. It seeks to transform an image–caption pair into a coherent digital keepsake by coordinating coarse affect recognition, memory-category assignment, nostalgic image grading, and explanatory presentation. The output is an application artifact, not a clinical or psychometric assessment. In particular, the code's "nostalgia score" is a hand-designed index and has not been validated against human ratings or a standardized nostalgia scale.

1.3 Objectives: The repository implements the following objectives:

1. accept a photograph and caption through a browser interface and REST API;
2. estimate coarse visual and textual affect using lightweight, inspectable rules;
3. fuse the modality labels and caption cues into a memory category;
4. calculate an implementation-defined score from temporal, reminiscence, affect, and confidence-like cues;
5. select one of six analog-photo presets and generate a styled JPEG;

6. produce a category-conditioned narrative and explanation fields; and
7. expose optional memory history through a database or browser storage.

1.4 Scope of Work: The scope comprises the checked-in Python backend, React/Vite frontend, local browser fallback, SQLite-compatible persistence layer, dataset directories, GoEmotions analysis utilities, API tests, configuration, launch scripts, and project documentation. The work describes runtime behavior from source code. It does not claim that the bundled image or text datasets trained the application, that model names in settings correspond to loaded networks, or that an evaluation was performed on a held-out benchmark. External APIs, pretrained checkpoints, user studies, and deployment monitoring are outside the evidence available in the repository.

1.5 Contributions: The implementation contributions are: (i) an end-to-end browser/server workflow linking caption context with photographic styling; (ii) a modular rule-based pipeline whose intermediate categories and score factors can be inspected; (iii) six configurable analog-inspired pixel-processing presets; (iv) an API and persistence design for archived memory records; and (v) an offline browser fallback. These are system-engineering contributions. The repository does not establish a new learned algorithm or performance advantage over prior methods.

2. Literature Review

2.1 Existing Methods: Emotion-analysis methods range from lexicons and hand-coded rules to supervised classifiers and contextual language models. Lexicons are easy to inspect and require little compute, but they are sensitive to vocabulary choice, context, negation, and domain shift. Transformer-based systems can represent context more flexibly but require model artifacts, training or inference dependencies, and appropriate evaluation. For images, facial-expression models generally operate on detected faces and learned visual features; global color statistics are cheaper but are strongly affected by lighting, white balance, and scene content.

Image styling similarly ranges from fixed filters and color transforms to neural style-transfer approaches [9]. Fixed filters are efficient and preserve scene structure but cannot synthesize semantic content. Multimodal fusion may occur at feature, representation, or decision level [6]. MemoriaAI uses late, rule-based decision fusion: it combines discrete labels and caption substring overrides rather than learned embeddings or cross-attention. SHAP [7] and Grad-CAM [8] are established model-attribution approaches, but this project's explanation features are approximations designed for interface display and must not be interpreted as their algorithmic outputs.

2.2 Related Work: GoEmotions provides 58,009 English Reddit comments labeled for 27 emotion categories and Neutral [4]. Its fine-grained taxonomy offers a useful resource for text-emotion research, but the project's runtime analyzer maps selected words to six coarse classes and does not load a GoEmotions-trained classifier. The included `calculate_metrics.py` can calculate thresholded multilabel metrics when given test labels and prediction files; no such prediction file or result JSON is included.

FER-2013 is a recognized facial-expression benchmark [5]. The project contains folders named for seven emotion classes, but only four classes are present under the current `train` directory and all seven appear under `test`. These folders are not used by the runtime image analyzer. Nostalgia research provides conceptual motivation for social, temporal, and bittersweet cues [1]–[3], but it does not validate the project's particular lexical weights, score caps, or style mappings.

2.3 Research Gaps: The repository exposes four gaps relevant to a research extension. First, there is no application-specific trained model despite documentation naming DeepFace and DistilRoBERTa. Second, the dataset files are not tied to a training/evaluation pipeline, and the image training split is incomplete. Third, the numerical score and explanations lack criterion, calibration, and faithfulness evaluations. Fourth, there is no controlled comparison or user study establishing whether the selected style is perceived as emotionally congruent or nostalgic. Thus the defensible research contribution is a functioning application prototype and a clear path to evaluation, not a measured improvement over existing methods.

3. Proposed Methodology

3.1 System Overview: The user provides caption text (T) and image (I). Separate backend services estimate an image label and a text label. A fusion service maps their pair to a category and may replace that category using caption keywords. A nostalgia-scoring service combines regex-based temporal cues, lexical memory cues, an affect-pair rule, and confidence-like values. A style service selects one preset or honors an explicit valid override, transforms the image, and encodes a JPEG. Narrative and explanation services add output fields before the combined result is returned and optionally saved.

3.2 Architecture: The primary architecture comprises a React 18/Vite frontend, FastAPI routes, Python service modules, Pydantic request/response schemas, Pillow/NumPy image operations, SQLAlchemy ORM, and SQLite by default. An alternate client path runs when the frontend health check cannot reach `http://localhost:8000`; it uses JavaScript heuristics, Canvas styling, and `localStorage`.

Fig. 1. System architecture. The diagram should show the user image and caption entering the React client; two parallel branches for RGB-statistic visual scoring and keyword-based text scoring; a rule-fusion node; downstream score, style, story, and explanation modules; then a results dashboard and optional persistence. Draw the API/backend path and browser fallback as distinct lanes because their logic and storage are not identical.

3.3 Workflow: The end-to-end API route `/process-memory` executes eight stages: (1) validate non-empty image and caption; (2) decode the image and derive visual scores; (3) tokenize and score caption keywords;

(4) fuse labels and caption context; (5) calculate the nostalgia index; (6) select and apply a photographic preset; (7) generate story and explanation payloads; and (8) attempt to persist a `MemoryRecord`. The response returns the original and transformed image data, intermediate labels, score factors, style metadata, story, explanation, ID, and timestamp. Separate endpoints expose each core stage, while history routes list or delete records.

3.4 Algorithms and Models: **Visual affect.** Pillow computes RGB means and standard deviations. Brightness is the arithmetic mean of RGB means; warmth is ((\mu_R-\mu_B)/\max(B,1)); contrast is the average channel standard deviation. Fixed initial scores are adjusted by threshold rules and normalized. The optional `deepface` import is only checked in the constructor; no DeepFace inference call occurs. On exceptions, a default Happy response is returned.

Text affect. The analyzer lowercases text and extracts alphabetic tokens. A hand-authored dictionary maps tokens to Joy, Sadness, Gratitude, Excitement, Love, and nostalgia cues. Exact token matches receive fixed increments; several phrase conditions add further boosts. The largest normalized score is returned as the label. This output is a normalized heuristic score, not a calibrated probability.

Fusion and scoring. A dictionary maps selected image/text label pairs to category names, followed by caption substring overrides for friend/reunion, family, and childhood cues. A confidence-weighted synergy value is returned. The nostalgia score uses a base value plus at most one matched temporal pattern, summed reminiscence weights, a rule-based resonance term, and a confidence term. A special case sets the result to 91 when the caption contains both "5 years" and "together again."

Style and explanations. Six presets use channel multipliers and offsets, contrast/saturation adjustments, optional blur blending, Gaussian noise, and a radial vignette. The returned diffusion prompt is descriptive metadata; no diffusion model is run. The explanation service uses fixed token weights and fixed contribution splits. Its heatmap consists of two Gaussian peaks at predetermined relative coordinates, color mapped and blended with the image. Neither SHAP nor gradient-based Grad-CAM is implemented.

3.5 Mathematical Formulation: Let channel means be (\mu_R,\mu_G,\mu_B), standard deviations (\sigma_R,\sigma_G,\sigma_B), brightness (B), warmth (W), and contrast (C):

[B=\frac{\mu_R+\mu_G+\mu_B}{3},\quad W=\frac{\mu_R-\mu_B}{\max(B,1)},\quad C=\frac{\sigma_R+\sigma_G+\sigma_B}{3}.]

Threshold rules map ((B,W,C)) to an uncalibrated visual score vector (\mathbf{s}_v), normalized as (p_v(k)=s_v(k)/\sum_j s_v(j)). For text, initialized class weights are incremented by matched keyword and phrase rules, giving (p_t(k)=s_t(k)/\sum_j s_t(j)). The predicted labels are (e_v=\arg\max_k p_v(k)) and (e_t=\arg\max_k p_t(k)). These normalized values are not learned posterior probabilities.

Fusion synergy is

[\Psi=0.45c_v+0.55c_t,]

where (c_v,c_t) are the top normalized heuristic scores. The score is the rounded, bounded sum

[S=\operatorname{round}{\min[98,\max(45,T+D+R+Q)]},\qquad Q=15\frac{c_v+c_t}{2},]

where (T) is the capped temporal-cue value, (D) the capped reminiscence-density value, and (R) the rule-based affect-resonance value. The implemented score range is normally 45–98, not 0–100. The hard-coded benchmark override supersedes this equation for its specific caption pattern.

For image styling, the selected preset applies channel transforms of the general form

[I'_c(x,y)=\operatorname{clip}(a_c I_c(x,y)+b_c,0,255),]

followed by contrast/saturation operations, optional blur blending, noise (\epsilon(x,y)\sim\mathcal{N}(0,\sigma^2)), and radial mask multiplication (V(x,y)). The final image is encoded as JPEG at quality 92. Grain is unseeded, so repeated outputs may differ.

4. Dataset and Data Preprocessing

4.1 Dataset Description: The repository contains the following research assets. Their presence is documented separately from their runtime use.

- GoEmotions: `train.tsv` contains 43,410 rows, `dev.tsv` 5,426, and `test.tsv` 5,427, alongside raw CSV annotations and emotion mappings. The backend does not use these assets for inference.
- FER-style images: train/test folders are present for seven named expressions, but only angry, disgust, fear, and happy training folders are present. The backend does not use these images for inference.
- Dataset analysis: GoEmotions analysis, word extraction, plots, mapping files, and a metric calculator are repository utilities; the API pipeline does not invoke them.

4.2 Data Collection: No data-collection process is implemented for application users beyond accepting their submitted image and caption. The repository contains public dataset files and supporting documentation. The app does not collect research annotations, consent, demographic metadata, or preference ratings. The GoEmotions README describes Reddit comments and annotator-agreement filtered splits; the project itself does not document a new collection or license audit.

4.3 Data Cleaning: At runtime, image input is base64-decoded and converted to RGB. The styling service downsizes images whose longest side exceeds 1600 pixels. Text is stripped, lowercased for matching, and tokenized with a regular expression that retains alphabetic characters and apostrophes. There is no dataset cleaning pipeline in the backend. The included GoEmotions preprocessing artifacts are upstream dataset resources, not steps executed by MemoriaAI at inference.

4.4 Feature Engineering: Visual features are global channel means, standard deviations, derived brightness, warmth, and contrast. They contain no face crop, landmark, embedding, or spatial feature map. Text features are binary/exact token

matches, substring phrase conditions, and counts of selected nostalgia cues. The fusion layer receives only top labels and confidence-like values plus the raw caption; it does not concatenate learned feature vectors or compute cross-attention.

4.5 Data Splitting: No split is created for the application, and no model is trained against the included datasets. GoEmotions supplies train/development/test files as described above. The FER-style directory naming suggests train/test partitions, but the training partition lacks three of the seven test labels. Therefore, the repository cannot support a complete seven-class supervised experiment without data repair and a validation split. The present results section does not use these folders as an evaluation set.

5. System Design and Implementation

5.1 Overall Architecture: The React single-page client handles upload, result visualization, style controls, and fallback history. It checks backend health and sends JSON to FastAPI when available. FastAPI routes validate schemas and call service modules. The combined pipeline assembles Pydantic response models, then attempts a database write. If the service is unavailable, the client-side routine classifies by caption substrings, applies Canvas effects, and stores up to 30 local records.

Fig. 2. End-to-end workflow. A flowchart should depict input validation, image/text cue extraction, fusion, score computation, preset selection and rendering, story/explanation generation, API response, and optional history persistence. Mark the client-side fallback with a separate dashed branch.

5.2 Module Description:

`image_emotion.py` RGB-statistic heuristic, image decoding, and exception fallback.

`text_emotion.py` Tokenization, emotion lexicons, phrase boosts, and polarity-like value.

`multimodal_fusion.py` Label-pair mapping, contextual overrides, and weighted synergy.

`nostalgia_scorer.py` Temporal regex, reminiscence lexicon, resonance, and score bounds.

`style_transfer.py` Six presets with color transforms, blur, grain, vignette, and JPEG output.

`story_generator.py` Category-conditioned narrative templates and a special caption case.

`explainability.py` Fixed keyword weights, contribution split, and synthetic Gaussian overlay.

`routes/` Component APIs, full memory workflow, and history operations.

`models/` and `database.py` Pydantic contracts and SQLAlchemy memory persistence.

- Frontend services and components: API health/fallback, Canvas operations, dashboard, and history views.

The SQLAlchemy model stores caption, image reference strings, labels and scores, selected style, narrative, explanation values, and timestamp. The processing route currently stores only the first 200 characters of base64 image fields, so history fields do not retain full image assets. Database save failure is caught and treated as nonfatal.

5.3 Technology Stack: The backend uses Python, FastAPI, Uvicorn, Pydantic/Pydantic Settings, SQLAlchemy, Pillow, and NumPy. Optional deep-learning dependencies listed in `requirements.txt` are commented out. The frontend uses React 18, Vite, Tailwind CSS, and browser Canvas; `localStorage` provides fallback history. SQLite is the default database URL, with a configurable SQLAlchemy URL. A Dockerfile uses Python 3.10 slim. Windows batch launchers support local startup. No database schema migration framework or trained-model serving component is present.

5.4 Implementation Details: The API exposes image/text affect, fusion, scoring, style, story, explanation, full memory processing, history, style listing, and health operations. The frontend health check times out after 1.8 seconds and selects a browser fallback when the backend is unavailable. Backend CORS currently allows all origins. The configured project settings include model identifiers and a `USE_DEEP_LEARNING_MODELS` flag, but the analysis services do not load those identifiers or use the flag. The diffusion-style strings returned by the style service are prompts only.

6. Experimental Setup

6.1 Hardware Configuration: No processor model, memory capacity, GPU, accelerator, or benchmark device is specified in the repository. Consequently, this paper makes no hardware-performance claim. The image operations use CPU-compatible Pillow and NumPy code, but actual runtime latency was not measured.

6.2 Software Environment: Backend requirements include FastAPI, Uvicorn, Pydantic, SQLAlchemy, Pillow, NumPy, SciPy, requests, pytest, and HTTPX. DeepFace, PyTorch, torchvision, and Transformers appear as optional commented dependencies. The frontend package declares React 18, Vite 5, Tailwind CSS 3, and supporting UI libraries. No lockfile or resolved environment report is present. The Dockerfile specifies Python 3.10 slim. These are declared dependencies, not a reproducible record of a completed training environment.

6.3 Hyperparameters: No model training occurs in the application, so there are no learned-model hyperparameters such as optimizer, learning rate, batch size, epochs, or training seed. The relevant hand-set runtime constants are six image base scores and threshold rules; text base weights and keyword increments; fusion coefficients 0.45/0.55; temporal and density weights/caps; score bounds 45–98; style defaults for warmth, grain, vignette, and fade; maximum styled image dimension 1600 pixels; JPEG quality 92; and a 1.8-second frontend health timeout. These are deterministic engineering parameters, not fitted hyperparameters.

6.4 Evaluation Metrics: The repository's API tests assert endpoint status, response fields, style availability, a specific hard-coded score, and non-empty narrative/explanation content. They are contract tests rather than predictive evaluation. The GoEmotions metric script can calculate accuracy, macro/micro/weighted precision, recall, and F1 at a configurable threshold (default 0.3) when predictions exist. No

prediction file, metric JSON, confusion matrix, timing result, or human-rating result is stored. Therefore, classifier metrics are unavailable and are not inferred from tests.

7. Results and Discussion

7.1 Experimental Results: The repository demonstrates implementation of an API-driven workflow and a browser fallback. The exact score of 91 for the test caption containing "5 years" and "together again" is enforced by a source-code special case and asserted in an API test. This confirms that the branch is present; it is not evidence of prediction accuracy or calibrated measurement. No empirical test set results are included.

- Accuracy: not measured for visual or text affect; not applicable to the hand-designed nostalgia score; style quality not measured.
- Precision, recall, and F1: not measured for either affect analyzer or style quality; not applicable to the nostalgia score.
- Confusion matrix: unavailable for the affect analyzers and style output; not applicable to the nostalgia score.
- Human preference or rating: not measured for any component.

7.2 Performance Analysis: The implementation is modular and uses conventional image operations without loading large neural weights. However, no latency, throughput, memory, or quality benchmark is included. Image grain is stochastic and can vary between runs. The same nominal workflow in the browser fallback differs from backend rules, so results may depend on whether the API is reachable. The application's output score should be read as a rule-derived index and not as a measured probability or clinical measure.

7.3 Comparative Analysis: No comparisons against DeepFace, transformer-based text classifiers, CLIP, neural style transfer, or alternative fusion systems are run. The README references several such methods, but the corresponding trained inference paths are absent. Accordingly, no superiority, improvement, or baseline delta is claimed. A meaningful comparison requires the same labeled evaluation set and a separate human evaluation of stylistic fit.

7.4 Error Analysis: No per-example predictions and ground-truth labels are available, so a quantitative error taxonomy cannot be computed. Source-level inspection identifies likely failure modes: exact keyword matching misses paraphrases and inflections; substring rules can trigger on negation or unrelated contexts; global warmth and brightness may describe illumination rather than facial affect; fallback labels may conceal invalid image inputs; and caption overrides can dominate contradictory visual evidence. The fixed Gaussian overlay can highlight empty regions because it does not detect faces. The story templates may not reflect the user's actual event. These are code-derived risks, not observed error rates.

7.5 Discussion: The system successfully integrates input handling, rule processing, image transformation, narrative display, API responses, and history interfaces. The primary strength is inspectability: intermediate values and mappings are explicit. Its scientific validity is limited by heuristic affect extraction, unvalidated score construction, incomplete image training folders, and lack of evaluation artifacts. The current explanation labels also overstate their technical basis if described as SHAP or Grad-CAM; accurate presentation should call them lexicon-based cue weights and a synthetic Gaussian visualization. The design is best positioned as a system prototype and baseline for subsequent validated research.

8. Advantages of the Proposed System

The implemented system offers a unified workflow that coordinates caption cues with image styling, rather than exposing only a standalone filter. Its modular service boundaries permit individual inspection and API reuse. The six presets provide distinct color treatments while preserving scene geometry. The local image pipeline avoids model downloads and is accessible on CPU-capable environments. The dashboard presents intermediate emotion, score, style, narrative, and explanation fields together. Backend history and the client-side fallback support continued use in both connected and offline demonstrations. These advantages describe software properties; they do not imply validated emotion recognition or user preference.

9. Limitations

The image analyzer does not detect faces or run DeepFace; it infers labels from global pixels and returns facial attributes heuristically. The text analyzer is a manually curated English lexicon and its normalized values are not calibrated probabilities. The fusion table and score weights are hand-authored, and the fixed 91 case introduces a benchmark-specific exception. The included FER-style train split is missing three classes, and neither dataset is connected to runtime inference. The repository contains no learned checkpoints, training record, measured metrics, or user study. "SHAP" and "Grad-CAM" terminology in product documentation does not match the implemented lexicon and Gaussian methods. Stories use fixed templates. Backend history truncates stored image data, CORS is permissive, and browser/backend fallback logic is not identical. Finally, the score has no demonstrated construct validity and should not be interpreted as a psychological measure.

10. Future Work

Future work should first complete and audit the FER-style dataset split and build a subject-independent training/validation/test protocol. A versioned face-emotion model and text classifier can then be integrated with model cards, checkpoint hashes, and documented preprocessing. GoEmotions experiments should report threshold selection and class-wise and macro/micro metrics, while a separate image/caption set should be annotated for memory context. The nostalgia index requires human-rated construct and criterion validation before numeric interpretation. Comparisons should include image-only, text-only, current rule fusion, and learned fusion. Explanations should be tied to actual prediction models and assessed for faithfulness. A user study can measure emotional fit, visual quality, trust, and narrative usefulness. Reproducibility

work should record hardware, dependency versions, seeds, raw predictions, and output artifacts. Privacy work should include consent, retention controls, complete asset handling, upload validation, and restricted CORS.

11. Conclusion

This paper analyzed MemoriaAI as implemented in the repository: a web system joining RGB-statistic image cues, keyword-based caption affect, rule fusion, a hand-designed nostalgia score, six photographic filters, template narratives, heuristic explanations, and optional memory history. The source code supports claims about this architecture and its output transformations. It does not support claims of trained emotion models, SHAP/Grad-CAM attribution, benchmark accuracy, or comparative superiority. The included datasets and metric utilities provide starting materials for research but are not part of the application's present training or evaluation. The system is therefore a useful implementation baseline whose scientific claims depend on future integration and controlled validation.

References

[1] C. Sedikides, T. Wildschut, J. Arndt, and C. Routledge, "Nostalgia: Past, present, and future," *Current Directions in Psychological Science*, vol. 17, no. 5, pp. 304–307, 2008, doi: 10.1111/j.1467-8721.2008.00595.x.

[2] C. Sedikides and T. Wildschut, "Finding meaning in nostalgia," *Review of General Psychology*, vol. 22, no. 1, pp. 48–61, 2018, doi: 10.1037/gpr0000109.

[3] T. Wildschut, C. Sedikides, J. Arndt, and C. Routledge, "Nostalgia: Content, triggers, functions," *Journal of Personality and Social Psychology*, vol. 91, no. 5, pp. 975–993, 2006, doi: 10.1037/0022-3514.91.5.975.

[4] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A dataset of fine-grained emotions," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 4040–4054, doi: 10.18653/v1/2020.acl-main.372.

[5] I. J. Goodfellow *et al.*, "Challenges in representation learning: A report on three machine learning contests," in *Neural Information Processing*, vol. 2, 2013, pp. 117–124, arXiv:1307.0414.

[6] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 2, pp. 423–443, 2019, doi: 10.1109/TPAMI.2018.2798607.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.

[9] L. A. Gatys, A. S. Ecker, and M. Bethge, "A neural algorithm of artistic style," *Journal of Vision*, vol. 16, no. 12, Art. no. 326, 2016, doi: 10.1167/16.12.326.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.

[11] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," in *Proc. 5th Workshop Energy Efficient Machine Learning and Cognitive Computing*, 2019, arXiv:1910.01108.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[14] S. Mohammad and P. Turney, "Crowdsourcing a word-emotion association lexicon," *Computational Intelligence*, vol. 29, no. 3, pp. 436–465, 2013, doi: 10.1111/j.1467-8640.2012.00460.x.

[15] P. Ekman, "An argument for basic emotions," *Cognition and Emotion*, vol. 6, no. 3–4, pp. 169–200, 1992, doi: 10.1080/02699939208411068.

[16] J. A. Russell, "A circumplex model of affect," *Journal of Personality and Social Psychology*, vol. 39, no. 6, pp. 1161–1178, 1980, doi: 10.1037/h0077714.

[17] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[18] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

Appendix

A. Figure Descriptions: **Fig. 1. System architecture.** Show the frontend, separate image and caption analyzers, rule-based fusion, score, style, story, and explanation services, response dashboard, and persistence. Label image and text analyzers "pixel-statistic heuristic" and "keyword heuristic."

Fig. 2. End-to-end workflow. Show validation, image/text analysis, fusion, scoring, preset selection, Pillow/NumPy rendering, story/explanation assembly, database attempt, and return. Indicate browser fallback as a parallel client-only route.

B. Repository Evidence and Reproducibility: The implementation is distributed across `backend/app/services/`, `backend/app/routes/`, `backend/app/models/`, and `frontend/src/`. Dataset assets reside in `image_emotion_dataset/` and `text_emotion_dataset/`. The backend test file checks API contracts and a special score branch; it does not evaluate dataset accuracy. No notebook, trained checkpoint, metrics result, confusion matrix, graph of measured application performance, or screenshot is included. Figures in a submission should therefore be generated from actual application runs and labeled as interface illustrations, not validation results.

For a reproducible future experiment, archive the repository revision, OS/CPU/GPU and memory, Python/Node versions, resolved dependencies, dataset provenance and complete split counts, model/checkpoint identifiers, seeds, predictions, metric scripts and outputs, and human-study protocol. The current manuscript deliberately does not invent missing measurements.